

Parallelizing Vampire via Semantically Entailed Starting Clause Sets

1 Motivation

Vampire’s main proving mode (superposition-based saturation) evolves a single global state from an initial clause set C_0 . This makes internal parallelization difficult, since inference generation, redundancy checks, and indexing are tightly coupled. Rather than parallelizing the saturation loop, we instead parallelize multiple independent saturation runs, each starting from a different but semantically related clause set.

2 Key Idea

Generate alternative starting clause sets $C_1, \dots, C_k$ such that $C_0 \models C_i$ for all i , and then run Vampire independently on each C_i in parallel. If any run proves $\text{UNSAT}(C_i)$, then $\text{UNSAT}(C_0)$ follows immediately, since unsatisfiability is preserved under entailment.

3 Semantic Reachability

Instead of requiring operational reachability via a specific sequence of superposition steps, we impose a semantic reachability contract:

a starting set C_i is acceptable if and only if $C_0 \models C_i$.

This avoids reasoning about prover internals and focuses purely on logical safety.

4 Structure of a Starting Set

Each candidate starting set is constructed as

$$C_i = B_i \cup S_i,$$

where $B_i \subseteq C_0$ is a base subset of original clauses and may be a strict subset of C_0 , allowing aggressive clause dropping. The set S_i is a small set of added seed clauses (lemmas, units, rewrite hints). Entailment reduces to checking that each seed clause is implied by C_0 .

5 Verifying Entailment

For a seed clause $s = (\ell_1 \vee \dots \vee \ell_m)$, entailment $C_0 \models s$ is checked by refutation:

$$C_0 \models s \iff C_0 \cup \{\neg \ell_1, \dots, \neg \ell_m\} \text{ is UNSAT.}$$

Vampire can be used directly for this check. Only candidates whose seeds are all certified are used for parallel proof search.

6 Role of an LLM or Intelligent Agent

An LLM is used only for hypothesis generation, not for proof or verification. It proposes non-obvious seed clauses or reformulations that are difficult to enumerate syntactically. All proposals are filtered by semantic checking, so soundness depends solely on Vampire.

7 High-Level Algorithm

1. Generate candidate forks (B_i, S_i) .
2. Certify each seed via entailment checking.
3. Run Vampire independently on each certified C_i .
4. If any run derives the empty clause, report UNSAT(C_0).

8 Why This Is Interesting

This approach enables parallelism without shared mutable state, preserves soundness by construction, allows exploration of diverse semantically valid reformulations, and is fully compatible with existing Vampire infrastructure.

9 One-Sentence Summary

We parallelize Vampire by launching independent saturation runs from clause sets semantically entailed by the original problem; candidate reformulations are generated heuristically and verified semantically before use.

10 Seed Clauses

A *seed clause* is a clause that is not explicitly present in the original clause set C_0 , but is logically implied by C_0 and injected into a modified starting set in order to bias proof search.

Intuitively, a seed clause represents a lemma that the prover might eventually derive on its own, but whose early availability can dramatically change the behavior of saturation. Seed clauses do not add logical strength; instead, they influence rewriting, clause selection, and redundancy elimination.

Formally, a clause s is a seed clause if:

- $s \notin C_0$,
- $C_0 \models s$,
- s is added to some starting set $C_i = B_i \cup S_i$.

Seed clauses are independently verified by semantic entailment checking before use, ensuring soundness.

11 Canonical Seed Clause Examples

Not all entailed clauses are useful as seeds. Experience with superposition provers suggests that small, structurally meaningful clauses have the highest payoff. Below is a non-exhaustive catalogue of canonical seed patterns.

11.1 Equality Closure and Rewrite Seeds

These seeds collapse multi-step rewriting into a single step.

- **Transitive equality:**

$$f(x) = g(x), g(x) = h(x) \Rightarrow f(x) = h(x)$$

- **Ground equality shortcut:**

$$f(a) = g(a), g(a) = h(a) \Rightarrow f(a) = h(a)$$

- **Rewrite orientation normalization:**

$$x = f(x) \rightsquigarrow f(x) = x$$

This represents a choice of orientation under an ordering or simplification strategy, rather than a new logical consequence.

These seeds enable immediate rewriting and often eliminate large families of inferences.

11.2 Unit Predicate Consequences

Unit clauses are particularly effective due to their strong simplifying effect.

- From $\neg P(x) \vee Q(x)$ and $P(a)$, add:

$$Q(a)$$

- From $\neg Q(a)$ and $P(x) \vee Q(x)$, add:

$$P(a)$$

Such seeds act as a form of first-order unit propagation.

11.3 Structural or Relational Seeds

These make implicit structure explicit.

- **Transitivity:**

$$\neg R(x, y) \vee \neg R(y, z) \vee R(x, z)$$

- **Symmetry:**

$$\neg R(x, y) \vee R(y, x)$$

When entailed, these clauses can significantly prune search.

11.4 Functional Consistency Seeds

These encode determinism or injectivity properties.

- **Injectivity-style:**

$$f(x) = f(y) \Rightarrow x = y$$

- **Determinism:**

$$P(f(x), y), P(f(x), z) \Rightarrow y = z$$

These are powerful but should be used sparingly and only when easily verifiable.

12 Minimal Seed Grammar for Prompting

To keep entailment checking tractable and avoid degrading saturation, seed clauses should be drawn from a restricted syntactic fragment. The following grammar defines a minimal, practical seed language with an intentional bias toward Horn-style or guarded clauses.

Terms

$$t ::= x \mid a \mid f(t_1, \dots, t_n)$$

Atoms

$$A ::= P(t_1, \dots, t_n) \mid t_1 = t_2 \mid t_1 \neq t_2$$

Seed Clauses

$$s ::= A \mid \neg A \mid A_1 \vee A_2 \mid \neg A_1 \vee A_2$$

Recommended Restrictions

- Prefer unit clauses (A or $\neg A$).
- Allow binary clauses only if they are guarded implications.
- Avoid clauses with more than two literals.
- Avoid deep term nesting.
- Avoid introducing new function or predicate symbols.

This grammar is expressive enough to capture most high-value seed patterns while keeping semantic verification cheap and predictable.

13 Interpretation

Seed clauses form the interface between speculative hypothesis generation and sound proof search. An external agent (e.g., an LLM) proposes candidate seeds within the grammar above; Vampire verifies entailment and exploits the resulting reformulation during saturation. This separation cleanly decouples creativity from correctness.

14 Related Work

Our proposal connects three research threads in automated reasoning that have largely been explored independently: parallel and portfolio-style theorem proving, splitting and SAT/SMT integration for managing search branching, and learning- or conjecture-driven guidance for proof search.

14.1 Parallelism in saturation-based theorem proving

Parallelizing saturation-based provers is well known to be difficult due to the tightly coupled global search state: active and passive clause sets, term indices, rewriting, and redundancy elimination interact in complex ways. As a result, most successful forms of parallelism in first-order automated theorem proving are coarse-grained rather than fine-grained.

A widely adopted approach is *portfolio parallelism*, where multiple prover instances or strategies are run in parallel and the first successful result is taken [1, 2]. This approach avoids shared-state synchronization and has proven highly effective in practice, especially in competition settings. Our method is closest in spirit to portfolio approaches, but differs in that each parallel run starts from a semantically certified reformulation of the same problem rather than merely a different heuristic configuration.

There has also been research on shared-clause or multi-threaded saturation architectures, where threads exchange learned clauses or coordinate given-clause selection. While promising, such approaches face significant engineering challenges and subtle completeness and redundancy issues. Our proposal deliberately avoids modifying the internal saturation loop, instead operating entirely at the level of problem reformulation.

14.2 Splitting and SAT/SMT integration

Splitting is a classical technique for introducing explicit branching into first-order proof search. Modern provers often combine splitting with propositional reasoning, allowing a SAT solver to manage Boolean structure while a first-order engine reasons under assumptions.

A prominent example is Vampire’s AVATAR architecture [3], which maintains a propositional abstraction of the clause set and uses SAT solving to guide and prune first-order reasoning across branches. AVATAR provides a principled and effective way to partition search space within a single run.

Our approach is complementary: instead of branching on logical structure inside one run, we generate alternative starting clause sets and explore them in parallel as independent runs. The semantic entailment contract $C_0 \models C_i$ plays a role analogous to SAT-level consistency in AVATAR, but at the level of whole problem reformulations. Investigating interactions between AVATAR-style splitting and semantically entailed forks is a promising direction for future work.

14.3 Learning-guided proof search and lemma generation

Learning-guided automated theorem proving has made substantial progress in recent years. Systems such as MaLAREa [4] demonstrated the effectiveness of learning-based premise selection over large libraries, while more recent approaches integrate machine learning directly into the prover to guide clause selection and search heuristics, for example ENIGMA-style guidance in saturation provers [5].

These approaches typically bias the *internal* search process of the prover, for example by scoring clauses for selection, rather than modifying the initial clause set. In contrast, our method operates by altering the starting point of proof search through additional, semantically verified clauses.

Lemma invention and conjecturing have also been studied in both automated and interactive theorem proving, though they are notoriously difficult to automate reliably. Our approach can be seen as a generate-and-validate scheme: speculative lemmas are proposed by an external agent and accepted only after independent semantic verification by the prover. This mirrors patterns seen in other areas of formal methods, such as counterexample-guided inductive synthesis and translation validation.

Recent successes of large language models in interactive theorem proving and program synthesis suggest that such models can act as powerful hypothesis generators. In our setting, LLMs are not trusted for correctness; they merely propose candidate seeds. Soundness is ensured entirely by semantic entailment checking performed by Vampire.

14.4 Certified problem transformations

Finally, the semantic reachability condition $C_0 \models C_i$ can be viewed as a form of certified problem transformation. Unlike traditional preprocessing pipelines, which rely on implementation correctness alone, our approach validates each transformation semantically. This yields a clean correctness argument and allows aggressive, speculative reformulations without compromising soundness.

In summary, our work combines ideas from portfolio parallelism, SAT-guided splitting, and learning-guided proof search into a unified workflow where alternative starting clause sets are generated speculatively, filtered by a semantic entailment contract, and explored in parallel by unmodified theorem prover instances.

References

- [1] G. Sutcliffe, “The tptp problem library and associated infrastructure,” *Journal of Automated Reasoning*, vol. 59, no. 4, pp. 483–502, 2017.
- [2] A. Riazanov and A. Voronkov, “The design and implementation of VAMPIRE,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2002, pp. 490–495.
- [3] L. Kovács and A. Voronkov, “First-order theorem proving and AVATAR,” in *Computer Aided Verification (CAV)*, 2013, pp. 1–18.
- [4] J. Urban, “Malarea: A metasystem for automated reasoning in large theories,” in *CADE*, 2007, pp. 1–15.
- [5] J. Jakubův and J. Urban, “Enigma: Efficient learning-based inference guiding machine,” in *CADE*, 2017, pp. 292–302.